Análisis crítico de Redis como sistema distribuido y comparación con Valkey

Emanuel Pino Juliana Romina

1. Resumen

2. Introducción

Redis es un almacén de datos en memoria que permite trabajar con estructuras como cadenas y hashes mediante comandos enviados a un servidor. La documentación oficial presenta usos que incluyen base de datos, caché, motor de flujos y agente de mensajería; por ello, Redis puede atender necesidades distintas de almacenamiento y acceso a datos, según la forma en que se configure y utilice [1], [2].

En este trabajo, Redis se estudia como sistema distribuido y se toma Valkey como sistema de comparación. El análisis se centra en Redis Cluster: cómo reparte los datos entre nodos, cómo se comunican los clientes y qué mecanismos ofrece para continuar operando ante ciertos fallos. Esta delimitación permite describir las capacidades documentadas del sistema sin asumir que una instalación independiente de Redis tiene las mismas propiedades que un clúster.

Como antecedentes, se han publicado tanto una propuesta de caché distribuida basada en Redis como una evaluación de Redis frente a MongoDB y Cassandra con cargas de YCSB. Sus resultados corresponden a los diseños y condiciones evaluados en cada artículo, por lo que no se consideran garantías generales sobre el rendimiento de Redis [3], [4].

3. Arquitectura

Redis puede ejecutarse como una instancia independiente. Para distribuir un conjunto de datos entre varios servidores, Redis Cluster organiza instancias Redis en una topología de nodos que asignan y atienden partes del espacio de claves. El clúster proporciona fragmentación de datos y puede

mantener operaciones ante algunos fallos, pero su disponibilidad depende de que permanezca accesible la mayoría de los nodos primarios y haya réplicas para los primarios que dejen de estar disponibles [5], [6].

3.1. Nodos primarios, réplicas y distribución de datos

En Redis Cluster, cada nodo primario es responsable de un subconjunto de las 16 384 ranuras de hash (*hash slots*). La ranura de una clave se calcula aplicando CRC16 a la clave y tomando el resultado módulo 16 384. También se pueden usar etiquetas de hash (*hash tags*) entre llaves para que claves relacionadas se asignen a la misma ranura. Esto permite que ciertas operaciones con varias claves se ejecuten cuando todas pertenecen a una misma ranura [6].

Un primario puede tener una o más réplicas. Las réplicas reciben los cambios del primario y, si este falla, una réplica elegible puede ser promovida para atender las ranuras que estaban asignadas a aquel. La replicación es asíncrona: el primario puede responder a una escritura antes de que todas sus réplicas la hayan recibido. Por ese motivo, la documentación advierte que una escritura reconocida por el primario puede perderse durante algunos escenarios de fallo [5], [6], [7].

3.2. Comunicación entre clientes y nodos

Los clientes se comunican con Redis mediante RESP, un protocolo de solicitud y respuesta en el que el cliente envía comandos y el servidor devuelve respuestas. La conexión suele realizarse por TCP al puerto de servicio de Redis, cuyo valor predeterminado es 6379. En un clúster, el cliente calcula o consulta qué nodo atiende la ranura de la clave. Los nodos no actúan como proxy de las solicitudes: si un cliente contacta al nodo equivocado, este puede responder con una redirección `MOVED` o, durante una migración de ranura, `ASK`. Los clientes compatibles pueden actualizar su mapa de ranuras y conectarse directamente al nodo correspondiente [6], [8].

La comunicación entre los nodos del clúster es independiente del protocolo de cliente. Los nodos se conectan mediante el bus del clúster, que usa un protocolo binario para intercambiar mensajes de estado, detección de fallos y cambios de configuración. De forma predeterminada, el puerto del bus se obtiene sumando 10 000 al puerto de servicio; por ejemplo, al puerto de servicio 6379 le corresponde el puerto 16379 [5], [6]. Un estudio de parcheo en vivo para Redis Cluster describe el uso de su protocolo interno de gossip para distribuir parches entre nodos; este trabajo examina la actualización del software, no modifica las garantías de replicación de datos [9].

3.3. Replicación y recuperación ante fallos

La revisión de Sabaghian y colaboradores explica que replicar datos puede mejorar la disponibilidad y la tolerancia a fallos, pero también exige decidir cuántas copias mantener y dónde ubicarlas [10]. En esta práctica, esas decisiones se reflejan en la distribución de los primarios y sus réplicas. Esto ayuda a entender por qué la recuperación depende de la topología elegida.

Durante el funcionamiento normal, el primario propaga a sus réplicas un flujo de comandos que refleja los cambios del conjunto de datos. Si se interrumpe la conexión, la réplica intenta reconectarse y solicitar una sincronización parcial de los comandos que no recibió. Si esto no es posible, se realiza una sincronización completa mediante una copia de los datos del primario y el posterior envío de los cambios recientes [7].

Redis Cluster intercambia mensajes de latido y usa información compartida entre nodos para detectar fallos. Un nodo puede marcar inicialmente a otro como posiblemente inaccesible; para considerar que un primario ha fallado y permitir una promoción, la condición debe ser confirmada por una mayoría de primarios dentro del intervalo establecido. Una réplica que cumple las condiciones puede solicitar votos y asumir el rol de primario. Si no es posible promover una réplica o no se conserva la mayoría requerida, el clúster puede dejar de atender solicitudes [5], [6].

Estos mecanismos no equivalen a una garantía de conservación de todas las escrituras confirmadas ni de disponibilidad ante cualquier partición. En particular, la replicación asíncrona deja una ventana en la que un primario puede fallar antes de que una escritura llegue a su réplica; además, el lado minoritario de una partición puede dejar de aceptar escrituras. Por tanto, la recuperación depende de la topología, de la conectividad entre nodos y de la configuración del clúster [6], [7].

La Figura 1 resume la topología descrita: los nodos primarios distribuyen las 16 384 ranuras de hash, cada uno cuenta con una réplica y los nodos se comunican mediante el bus del clúster. La leyenda distingue estas conexiones. El esquema es de elaboración propia y está basado en la documentación oficial de Redis [5], [6].

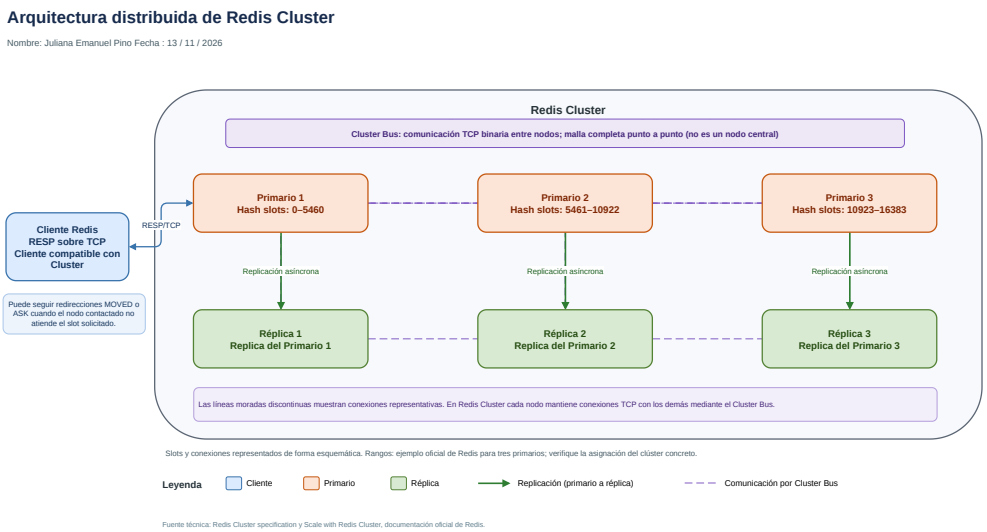


Figura 1: Arquitectura distribuida de Redis Cluster.

4. Transparencias

5. CAP y PACELC

5.1. Análisis de Redis Cluster según CAP

El teorema CAP describe una limitación que se manifiesta cuando hay una partición de red: un sistema distribuido no puede garantizar simultáneamente consistencia fuerte y disponibilidad para todas las solicitudes. No significa que cada sistema tenga que elegir de forma permanente dos propiedades y abandonar la tercera; la decisión relevante se presenta durante la partición [11].

En Redis Cluster, el resultado depende de qué nodos y ranuras quedan accesibles. El lado que conserva la mayoría de los primarios puede continuar si cuenta con réplicas elegibles para reemplazar a los primarios aislados. El lado minoritario puede aceptar escrituras durante una ventana limitada, pero, transcurrido el *node timeout* sin contacto con la mayoría, deja de aceptar consultas de escritura. Además, con la configuración predeterminada de cobertura completa, la pérdida de cobertura de alguna ranura puede hacer que el clúster rechace escrituras [5], [6].

Esta restricción de disponibilidad no equivale a una garantía de consistencia fuerte. La replicación entre un primario y sus réplicas es asíncrona, por

lo que un primario puede responder a una escritura antes de que una réplica la reciba. Si el primario falla y se promueve una réplica desactualizada, esa escritura confirmada puede perderse. Por ello, no es preciso clasificar Redis Cluster simplemente como CP o AP: durante una partición puede rechazar escrituras en el lado minoritario para limitar divergencias, pero no garantiza conservar todas las escrituras reconocidas [6], [7].

5.2. Análisis de Redis Cluster según PACELC

Datta y Oggier revisan los problemas de concurrencia y consistencia al modificar datos protegidos con códigos de borrado [12]. Aunque estudian una técnica de redundancia distinta de la replicación de Redis, su trabajo permite distinguir dos necesidades: mantener información redundante y coordinar sus actualizaciones. Al evaluar Redis Cluster, también es necesario revisar por separado la recuperación de datos y las garantías de lectura y escritura.

PACELC complementa CAP al distinguir el caso de partición (P), donde se contraponen disponibilidad (A) y consistencia (C), del funcionamiento normal (E), donde existe un intercambio entre latencia (L) y consistencia (C) [13].

En la rama P, Redis Cluster favorece la continuidad de las operaciones en el lado mayoritario cuando se cumplen las condiciones de quórum, cobertura de ranuras y disponibilidad de réplicas para el failover. En el lado minoritario limita las escrituras después del tiempo configurado. No obstante, las escrituras aceptadas antes de ese límite pueden perderse si el primario aislado es reemplazado por una réplica que no las recibió [5], [6].

En la rama E, el primario normalmente responde sin esperar confirmación de sus réplicas. Esta decisión reduce la espera de las escrituras, pero deja un intervalo en el que una falla puede causar pérdida de datos y permite que las réplicas tengan retraso respecto del primario. El comando `WAIT` permite esperar confirmaciones de réplicas y reducir la probabilidad de pérdida, aunque la documentación aclara que no convierte a Redis Cluster en un sistema de consistencia fuerte [5], [6].

6. Modelos de comunicación

7. Tolerancia a fallos

Kokociński, Kobus y Wojciechowski analizan sistemas replicados de alta disponibilidad considerando fallos de máquinas y particiones de red [14]. Su estudio muestra la importancia de incluir esos escenarios al evaluar las

garantías de consistencia. Por eso, las pruebas siguientes permiten observar el failover en nuestro entorno, pero no bastan para demostrar que todas las escrituras se conservan ante cualquier fallo.

Las pruebas se realizaron en una topología de seis nodos con tres primarios y tres réplicas. En la primera prueba se detuvo `redis-3`; según el estado observado durante el experimento, `redis-4` asumió el rol de primario para las ranuras que atendía su grupo. Al iniciar nuevamente `redis-3`, el listado de nodos lo mostró como réplica de `redis-4`. También se pudo consultar el dato de prueba después del failover. La Figura 5 documenta el regreso de `redis-3` como réplica.

Esta prueba es distinta de la segunda: se detuvo `redis-4` y `redis-3` asumió el rol de primario. En la captura correspondiente, `CLUSTER INFO` informó `cluster_state:ok` y los 16 384 slots asignados y disponibles. La Figura 6 muestra ese estado. Estos resultados describen únicamente las ejecuciones realizadas en este entorno; no garantizan la misma respuesta ante cualquier combinación de fallos o particiones.

8. Experimentación

Las pruebas se ejecutaron con los seis servicios Redis 7.4 del archivo Compose del experimento. La configuración solicitó tres primarios y una réplica para cada primario, dentro de una red de Docker propia. Las capturas siguientes son evidencia de las salidas observadas en la terminal.

8.1. Estado y distribución del clúster

La consulta de `CLUSTER INFO` indicó que el clúster estaba activo, con seis nodos conocidos y tres primarios. También informó que los 16 384 slots estaban asignados y disponibles, sin slots fallidos (Figura 2). En la salida de `CLUSTER NODES` se observaron los rangos 0–5460 en `redis-1`, 5461–10922 en `redis-2` y 10923–16383 en `redis-4`. Los nodos restantes aparecieron como sus réplicas (Figura 3).

8.2. Almacenamiento, consulta y asignación de una clave

Se almacenó el valor `Juliana` con la clave `estudiante` desde `redis-1`. Después, se consultó la misma clave desde `redis-2` con un cliente que sigue las redirecciones del clúster; la respuesta fue `Juliana`. La consulta `CLUSTER`

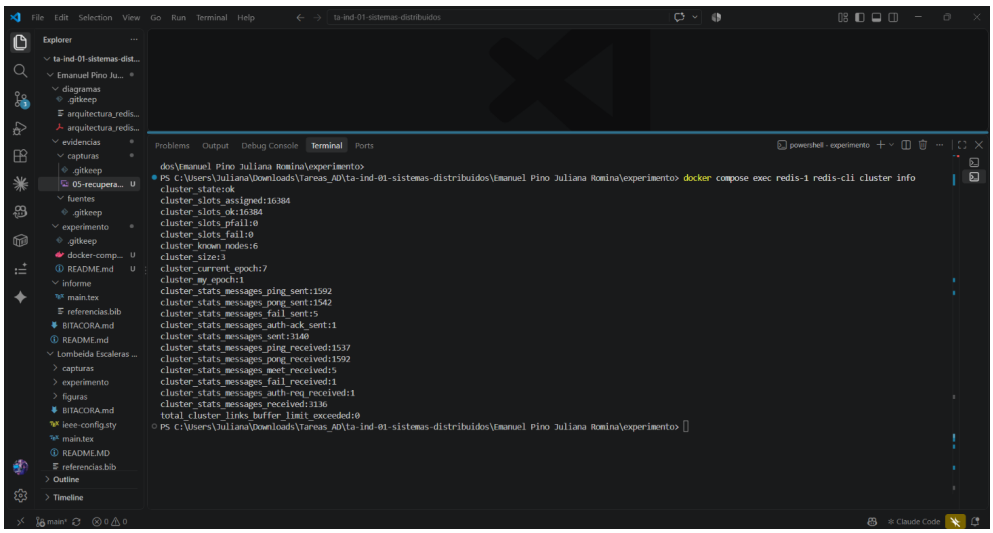


Figura 2: Estado del clúster: seis nodos conocidos, tres primarios y los 16 384 slots asignados y disponibles.

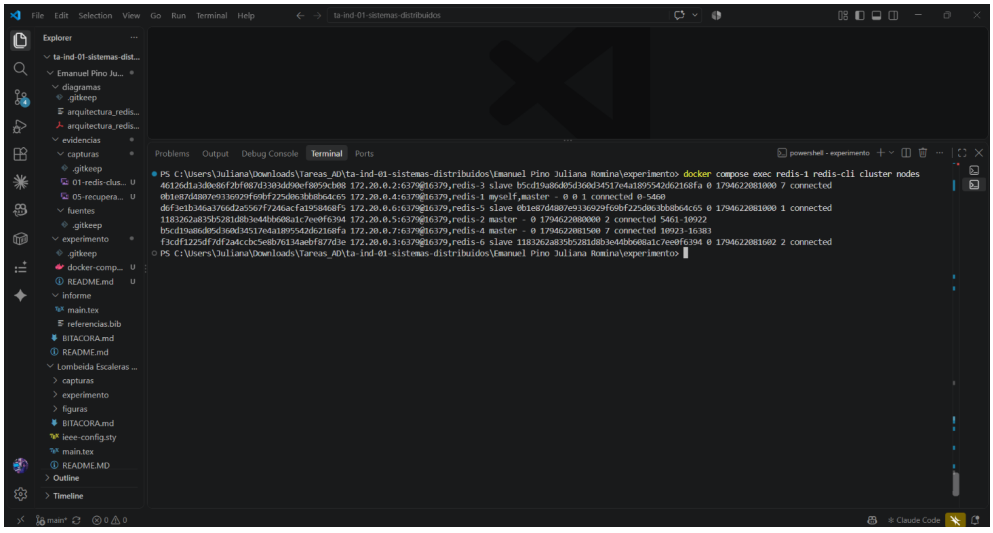


Figura 3: Salida de `CLUSTER NODES`: rangos de slots de los primarios y relación de sus réplicas.

`KEYSLOT estudiante` devolvió el slot 13012 (Figura 4). Este resultado confirma la asignación de esa clave en esta ejecución y la posibilidad de consultarla iniciando la conexión desde otro nodo.

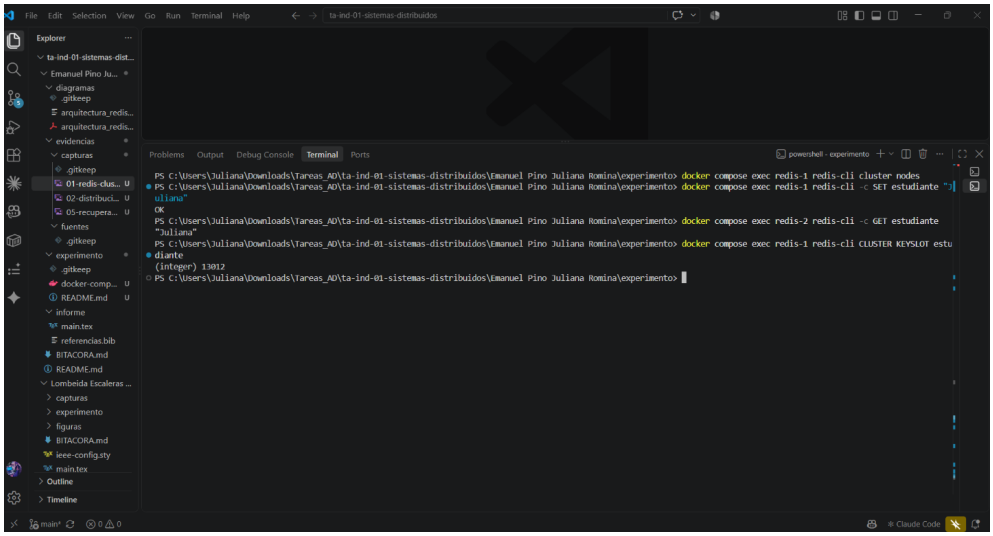


Figura 4: Escritura y lectura de `estudiante` desde nodos distintos; Redis informó que su slot es 13012.

8.3. Pruebas de failover y recuperación

En la primera prueba se detuvo `redis-3`. `redis-4` asumió el rol de primario y se pudo consultar el valor de prueba `prueba_failover`, cuya respuesta fue `Redis sigue funcionando`. Luego se inició nuevamente `redis-3`; la salida de `CLUSTER NODES` lo mostró como réplica de `redis-4` (Figura 5).

En una segunda prueba independiente se detuvo `redis-4`. La salida de `CLUSTER NODES` mostró a `redis-4` como fallido y a `redis-3` como primario. En ese momento, `CLUSTER INFO` mantuvo `cluster_state:ok`, informó los 16 384 slots asignados y disponibles, y registró seis nodos conocidos (Figura 6). Este segundo escenario no corresponde a la caída inicial de `redis-3`: se trata de una prueba separada en la que falló `redis-4`.

9. Ejemplos del entorno

Los siguientes comandos de `redis-cli` se ejecutaron desde los contenedores correspondientes mediante `docker compose exec`:

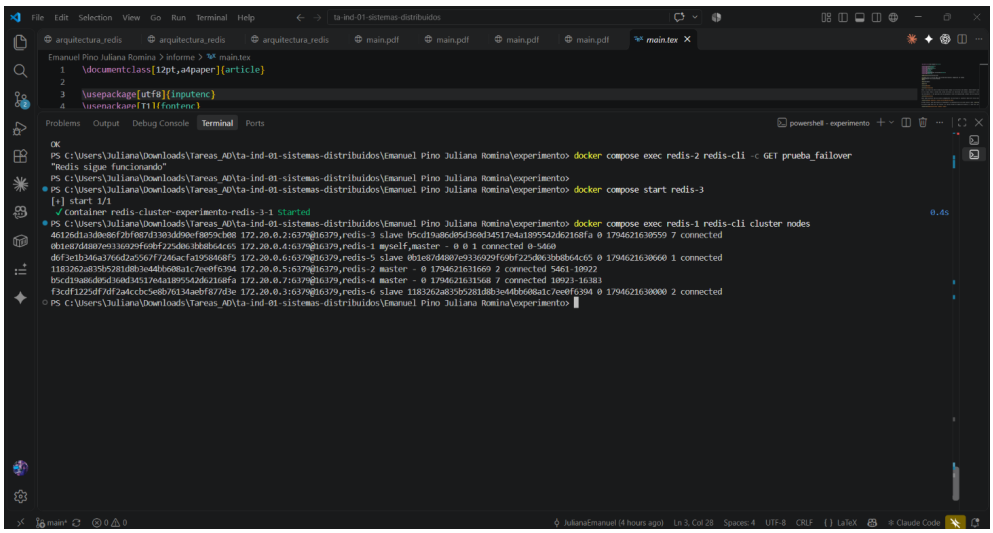


Figura 5: Tras iniciar de nuevo `redis-3`, el listado de nodos lo muestra como réplica de `redis-4`; también se observa la consulta del dato de prueba.

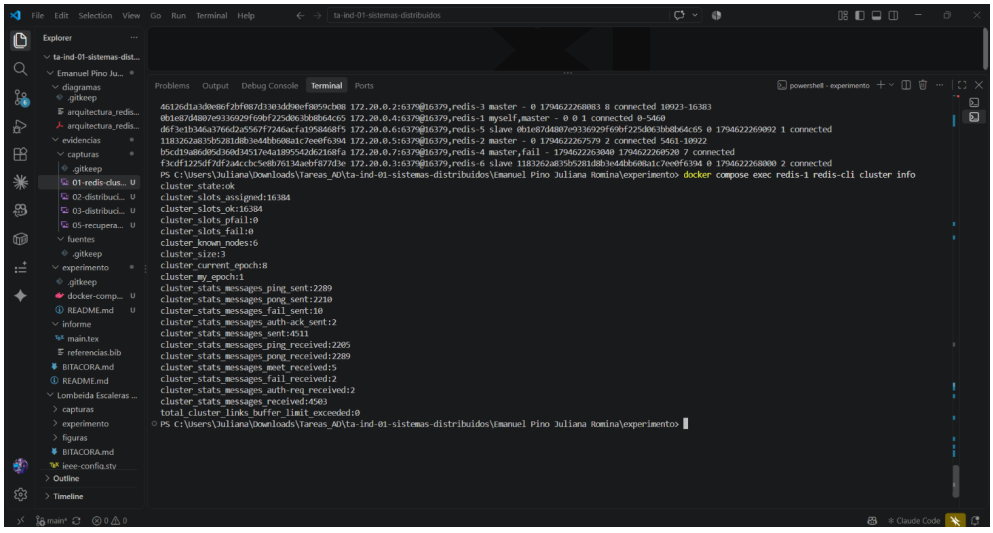


Figura 6: Segunda prueba de failover: `redis-4` aparece fallido, `redis-3` como primario y el clúster conserva la cobertura de slots.

```
redis-cli -c SET estudiante "Juliana"
redis-cli -c GET estudiante
redis-cli CLUSTER KEYSLOT estudiante
```

La opción `-c` permitió que el cliente siguiera las redirecciones de Redis Cluster al consultar desde un nodo distinto al que recibió la escritura.

10. Análisis comparativo: Redis Cluster y Valkey

Valkey es un almacén de datos en memoria de código abierto y un fork del código abierto de Redis. Surgió en 2024, después de que Redis Ltd. anunciara un cambio de licencia para Redis; la comunidad continuó el desarrollo bajo el nombre Valkey como proyecto de la Linux Foundation. Esta relación histórica explica que ambos proyectos compartan buena parte del diseño de Redis Cluster, aunque no implica que todas sus versiones y características sean idénticas [15].

La tabla muestra que, en las propiedades distribuidas solicitadas, las especificaciones actuales describen modelos muy parecidos. Esa similitud es coherente con el origen de Valkey como fork, pero no demuestra que sus implementaciones tengan el mismo comportamiento en toda versión o configuración. Tampoco permite concluir que uno sea más rápido o confiable: los artículos académicos citados en el análisis CAP y PACELC proporcionan marcos teóricos, no una comparación experimental directa entre Redis Cluster y Valkey [11], [13].

Desde una perspectiva crítica, ambos sistemas documentan el mismo tipo de compromiso: la replicación asíncrona reduce la espera de las escrituras, pero deja abierta la posibilidad de perder cambios reconocidos; y durante ciertas particiones se restringen las escrituras en el lado minoritario. Esto puede ser aceptable para aplicaciones que priorizan acceso de baja latencia y contemplan reintentos o pérdida limitada de datos. Para requisitos de consistencia fuerte o conservación estricta de cada escritura, la documentación por sí sola no basta para justificar la elección. También conviene evaluar el gobierno del proyecto, las licencias, las versiones, los clientes disponibles y las necesidades operativas. La elección entre ambos debe basarse en esos requisitos y en pruebas controladas que todavía no se han realizado.

Cuadro 1: Comparación de las características documentadas de Redis Cluster y Valkey Cluster.

Aspecto	**Redis Cluster**	**Valkey Cluster**
Origen y evolución	Proyecto Redis; Redis Cluster es su modo distribuido.	Fork de la base de código abierto de Redis, iniciado en 2024 como proyecto de la Linux Foundation. [15]
Arquitectura y particionamiento	Distribuye los datos entre primarios y asigna a cada nodo un subconjunto de 16 384 ranuras. Usa CRC16 módulo 16 384 y admite etiquetas de hash. [6]	También distribuye los datos entre primarios mediante 16 384 ranuras, CRC16 módulo 16 384 y etiquetas de hash. [16]
Comunicación	Los clientes usan RESP y se conectan al nodo responsable de la ranura. Las respuestas `MOVED` y `ASK` permiten redirigirlos; los nodos usan un bus binario del clúster. [6], [8]	Usa RESP para los clientes y redirecciones `MOVED` y `ASK`. Los nodos se comunican mediante su bus binario del clúster. [16], [17]
Replicación	Replicación primaria-réplica asíncrona. El primario puede confirmar una escritura antes de que llegue a sus réplicas. `WAIT` y `WAITAOF` permiten solicitar confirmaciones, pero no ofrecen consistencia fuerte. [5], [6], [7], [18]	Replicación primaria-réplica asíncrona. También ofrece `WAIT` y `WAITAOF` para esperar confirmaciones sin garantizar consistencia fuerte. [19], [20], [21]
Consistencia y tolerancia a fallos	La elección de una réplica depende de la detección de fallos y del quórum. Una escritura reconocida aún puede perderse durante un fallo o partición. [5], [6]	Emplea mecanismos equivalentes de detección, quórum y promoción de réplicas. También puede perder escrituras reconocidas; el lado minoritario deja de aceptar escrituras tras el *node timeout*. [16], [20]

11. Postura crítica

Redis Cluster ofrece un compromiso práctico entre rendimiento, escalabilidad horizontal y recuperación ante fallos, pero esas propiedades no deben confundirse con una garantía de consistencia linealizable o de disponibilidad ante cualquier partición. Su comportamiento depende de la topología, de que existan réplicas elegibles, de la mayoría de primarios accesible y de parámetros como `cluster-node-timeout` y `cluster-require-full-coverage` [5], [6].

Desde una perspectiva de ingeniería, este diseño puede ser adecuado cuando la aplicación tolera una ventana acotada de indisponibilidad de escritura en nodos aislados y puede aceptar el riesgo documentado de perder algunas escrituras confirmadas. En cambio, si cada escritura reconocida debe conservarse y observarse de manera estrictamente consistente, es necesario evaluar mecanismos adicionales y considerar un sistema con garantías distintas. Esta valoración se basa en las propiedades documentadas de Redis Cluster; no representa resultados de los experimentos que se desarrollarán más adelante.

12. Conclusiones

En la prueba, Redis Cluster distribuyó las ranuras entre tres primarios y mantuvo una réplica por cada uno. La clave `estudiante` se asignó al slot 13012 y pudo leerse desde otro nodo. Esto ayudó a relacionar el particionamiento del espacio de claves con las redirecciones que maneja un cliente compatible con el clúster.

En las pruebas de fallo, Redis Cluster promovió una réplica en cada uno de los dos escenarios realizados. Cuando se detuvo `redis-3`, `redis-4` pasó a ser primario y, al regresar, `redis-3` se incorporó como su réplica. En una segunda prueba, al detener `redis-4`, `redis-3` asumió el rol de primario y las salidas capturadas mostraron el clúster activo y con todos los slots disponibles. Estas observaciones corresponden a esta topología y a estas ejecuciones; no demuestran tolerancia ilimitada a fallos.

La práctica también hizo visible el papel de la replicación asíncrona: la promoción de una réplica permite continuar ante ciertos fallos, pero no significa que cada escritura confirmada ya estuviera replicada. Por eso, la disponibilidad observada debe analizarse junto con el quórum, la cobertura de slots y la posible pérdida de escrituras durante un fallo. La comparación teórica con Valkey se conserva en este informe, pero no se realizaron pruebas equivalentes con Valkey y, por tanto, no se presentan conclusiones experimentales de rendimiento entre ambos sistemas.

13. Declaración de uso de IA

14. Anexos

14.1. Evidencias bibliográficas

Las siguientes capturas documentan las fuentes académicas consultadas y su relación con el análisis de Redis Cluster.

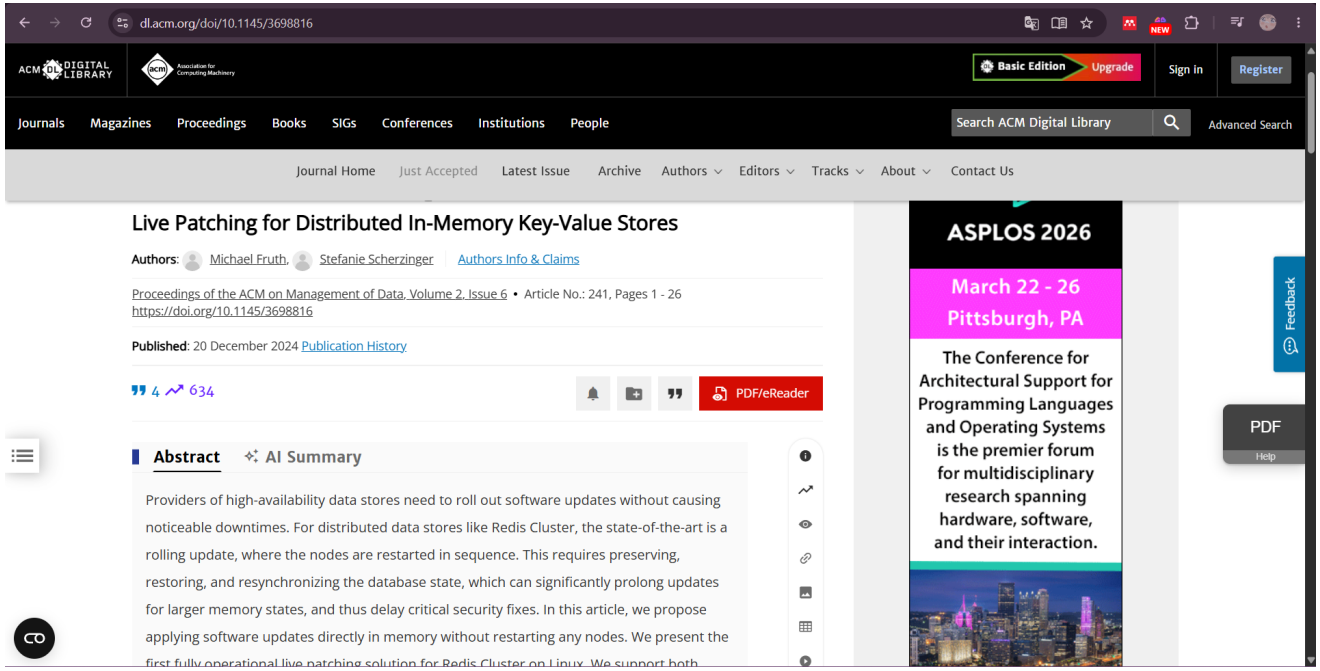
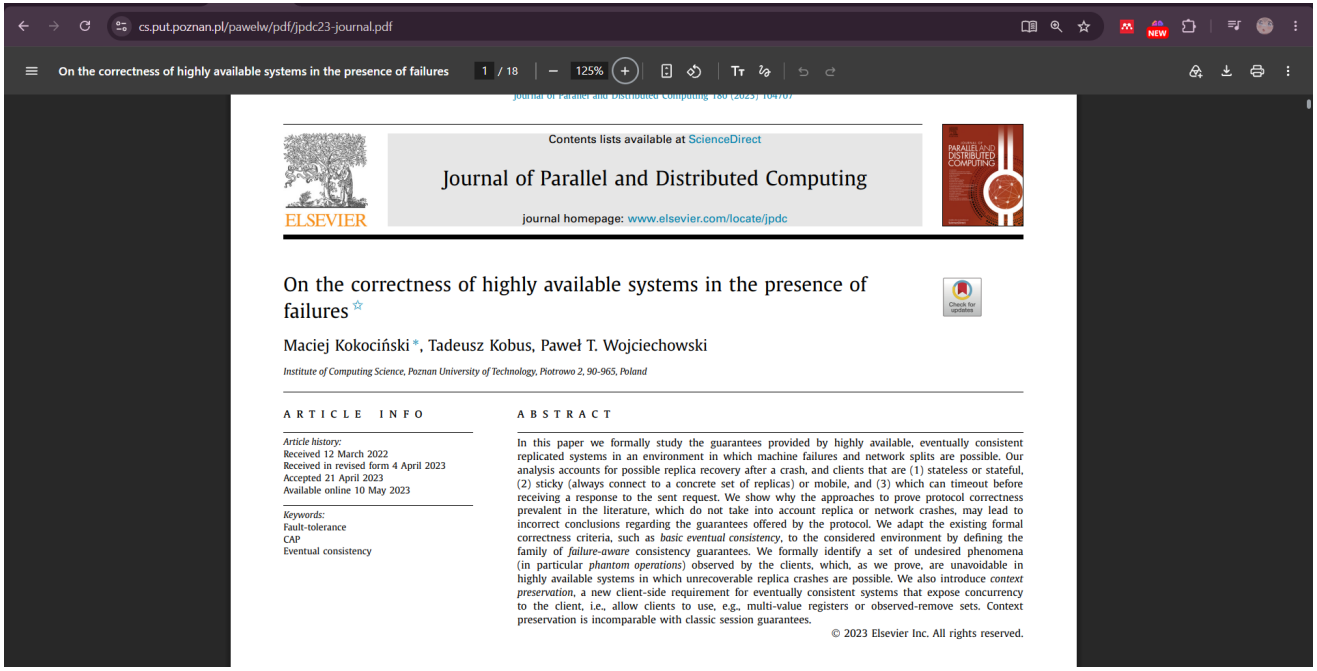


Figura 7: Ficha editorial y fragmento del resumen de *Live Patching for Distributed In-Memory Key-Value Stores* [9].

La captura identifica a los autores, el año, el DOI y el artículo 241. El fragmento visible explica que las actualizaciones sucesivas de Redis Cluster requieren reiniciar nodos y resincronizar su estado, y presenta la propuesta de aplicar parches en memoria sin reiniciarlos. Esta fuente relaciona el mantenimiento del clúster con la continuidad del servicio; no demuestra que todas las escrituras se conserven ante un fallo.



cs.put.poznan.pl/pawelw/pdf/jpdc23-journal.pdf

On the correctness of highly available systems in the presence of failures 1 / 18 125%

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

journal homepage: www.elsevier.com/locate/jpdc

ELSEVIER

On the correctness of highly available systems in the presence of failures ☆

Maciej Kokociński *, Tadeusz Kobus, Paweł T. Wojciechowski

Institute of Computing Science, Poznan University of Technology, Piotrowo 2, 90-965, Poland

ARTICLE INFO

Article history:
Received 12 March 2022
Received in revised form 4 April 2023
Accepted 21 April 2023
Available online 10 May 2023

Keywords:
Fault-tolerance
CAP
Eventual consistency

ABSTRACT

In this paper we formally study the guarantees provided by highly available, eventually consistent replicated systems in an environment in which machine failures and network splits are possible. Our analysis accounts for possible replica recovery after a crash, and clients that are (1) stateless or stateful, (2) sticky (always connect to a concrete set of replicas) or mobile, and (3) which can timeout before receiving a response to the sent request. We show why the approaches to prove protocol correctness prevalent in the literature, which do not take into account replica or network crashes, may lead to incorrect conclusions regarding the guarantees offered by the protocol. We adapt the existing formal correctness criteria, such as *basic eventual consistency*, to the considered environment by defining the family of *failure-aware* consistency guarantees. We formally identify a set of undesired phenomena (in particular *phantom operations*) observed by the clients, which, as we prove, are unavoidable in highly available systems in which unrecoverable replica crashes are possible. We also introduce *context preservation*, a new client-side requirement for eventually consistent systems that expose concurrency to the client, i.e., allow clients to use, e.g., multi-value registers or observed-remove sets. Context preservation is incomparable with classic session guarantees.

© 2023 Elsevier Inc. All rights reserved.

Figura 8: Primera página y resumen de *On the correctness of highly available systems in the presence of failures* [14].

Las primeras líneas del resumen describen el análisis de sistemas replicados con consistencia eventual ante fallos de máquinas y particiones de red. Este enfoque ayuda a interpretar las pruebas de Redis Cluster: observar una promoción de réplica no basta para establecer todas sus garantías de consistencia. El artículo aporta un marco general y no una evaluación específica de Redis o Valkey.

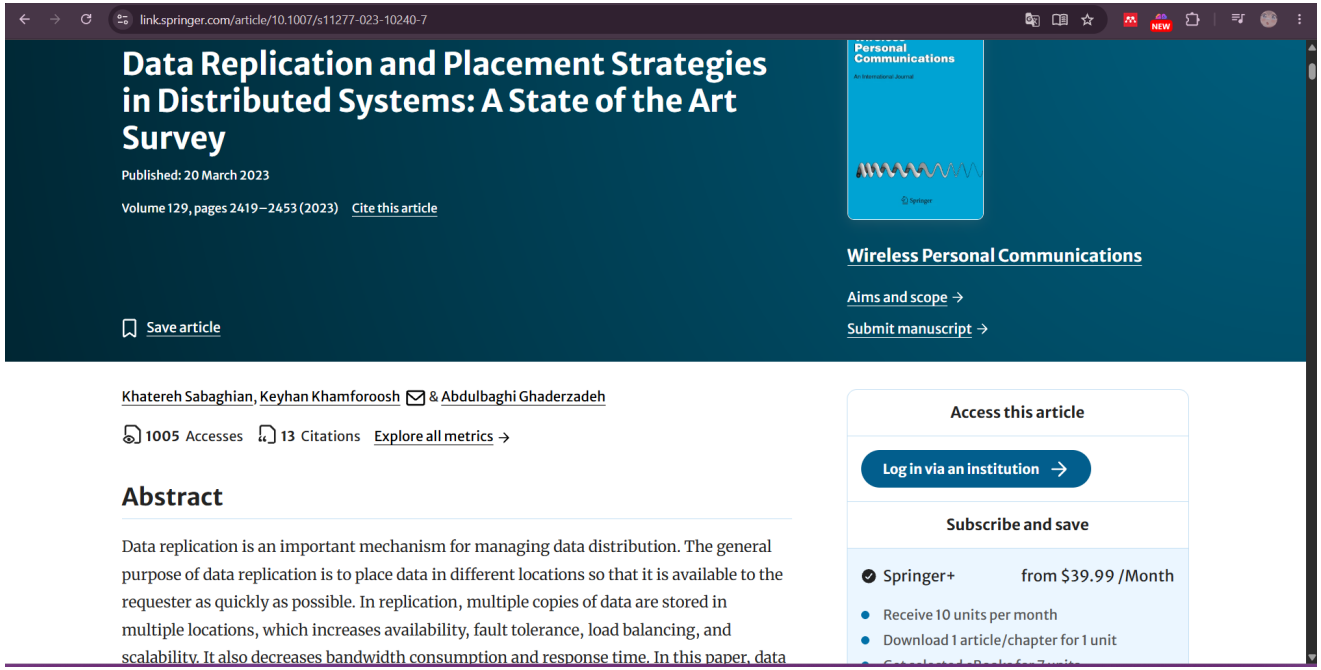


Figura 9: Ficha editorial y fragmento del resumen de *Data Replication and Placement Strategies in Distributed Systems: A State of the Art Survey* [10].

El fragmento visible relaciona el almacenamiento de copias en distintas ubicaciones con disponibilidad, tolerancia a fallos, balanceo de carga y escalabilidad. Esta revisión permite explicar el propósito de las réplicas en la topología del experimento. La mejora concreta depende de la estrategia empleada; la captura no demuestra que Redis Cluster proporcione consistencia fuerte ni disponibilidad ante cualquier fallo.

Referencias

[1] Redis. «Redis as an in-memory data structure store quick start guide,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/develop/get-started/data-store/

[2] Redis. «Quick starts,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/develop/get-started/

[3] L. Shi, H. Qiao, C. Yang, Y. Jiang, K. Yu y C. Chen, «Research and Application of Distributed Cache Based on Redis,» *Journal of Software*, vol. 19, n.º 1, págs. 1-8, 2024, doi: 10.17706/jsw.19.1.1-8.

[4] M. A. Kausar, M. Nasar y A. Soosaimanickam, «A Study of Performance and Comparison of NoSQL Databases: MongoDB, Cassandra, and Redis Using YCSB,» *Indian Journal of Science and Technology*, vol. 15, n.º 31, págs. 1532-1540, 2022, doi: 10.17485/IJST/v15i31.1352.

[5] Redis. «Scale with Redis Cluster,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

[6] Redis. «Redis cluster specification,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

[7] Redis. «Redis replication,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/

[8] Redis. «Redis serialization protocol specification,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/develop/reference/protocol-spec/

[9] M. Fruth y S. Scherzinger, «Live Patching for Distributed In-Memory Key-Value Stores,» *Proceedings of the ACM on Management of Data*, vol. 2, n.º 6, págs. 1-26, 2024, art. 241, doi: 10.1145/3698816.

[10] K. Sabaghian, K. Khamforoosh y A. Ghaderzadeh, «Data Replication and Placement Strategies in Distributed Systems: A State of the Art Survey,» *Wireless Personal Communications*, vol. 129, n.º 4, págs. 2419-2453, 2023, doi: 10.1007/s11277-023-10240-7.

[11] S. Gilbert y N. Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002, doi: 10.1145/564585.564601.

[12] A. Datta y F. Oggier, «Concurrency Control and Consistency Over Erasure Coded Data,» *IEEE Access*, vol. 10, págs. 118 617-118 638, 2022, doi: 10.1109/ACCESS.2022.3219875.

[13] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012, doi: 10.1109/MC.2012.33.

[14] M. Kokociński, T. Kobus y P. T. Wojciechowski, «On the correctness of highly available systems in the presence of failures,» *Journal of Parallel and Distributed Computing*, vol. 180, 2023, art. 104707, doi: 10.1016/j.jpdc.2023.04.008.

[15] Valkey. «History,» visitado 13 de nov. de 2026. dirección: https://valkey.io/topics/history/

[16] Valkey. «Cluster specification,» visitado 13 de nov. de 2026. dirección: https://valkey.io/topics/cluster-spec/

[17] Valkey. «Serialization protocol specification,» visitado 13 de nov. de 2026. dirección: https://valkey.io/topics/protocol/

[18] Redis. «WAITAOF,» visitado 13 de nov. de 2026. dirección: https://redis.io/docs/latest/commands/waitaof/

[19] Valkey. «Replication,» visitado 13 de nov. de 2026. dirección: https://valkey.io/topics/replication/

[20] Valkey. «Cluster tutorial,» visitado 13 de nov. de 2026. dirección: https://valkey.io/topics/cluster-tutorial/

[21] Valkey. «WAITAOF,» visitado 13 de nov. de 2026. dirección: https://valkey.io/commands/waitaof/